AI-FIRST SDLC PLATFORM

Platform Overview, Phase-by-Phase Build, & Local Execution Guide

An engine-agnostic, multi-tenant, web-delivered agentic platform that runs the full software lifecycle — ideation through deployment — on a client's real tools of record, with a human review gate at every state change.

Prepared for	The Engineering Team — EdvenSoft
Scope	Phases 0–4 (complete & tested); Phase 5 next
Document	Goal & overview · phase build descriptions · what works today end-to-end · hands-on local testing guide · setup & credentials
Date	June 29, 2026
Source of truth	/CLAUDE.md · /BUILD_PLAN.md · /docs/*

Contents

1. What We Are Building — and What We Are Trying to Achieve 3

1.1 The ultimate goal . . . 3

1.2 The seven non-negotiables (the rules the whole system obeys) . . . 3

1.3 The architecture at a glance — seven planes . . . 4

1.4 What “done” looks like at each milestone . . . 4

2. The Build, Phase by Phase (Phases 0–4) 5

2.1 Phase 0 — The Platform Spine . . . 5

2.2 Phase 1 — First System of Record (Atlassian) . . . 6

2.3 Phase 2 — Build Execution (real code, real PRs, durably) . . . 6

2.4 Phase 3 — Design & Test Loops . . . 7

2.5 Phase 4 — Brownfield-Full & Multi-Repo . . . 8

3. The Local Execution & Testing Guide 9

3.1 Part A — Bring up the platform (no external accounts needed) . . . 9

3.2 Part B — Drive the planning slice in the browser (Phase 0 end-to-end) . . . 10

3.3 Part C — Optional: exercise the live connectors . . . 11

3.4 Part D — See the cross-cutting guarantees with your own eyes . . . 12

4. Setup, Credentials & Prerequisites 13

4.1 Prerequisites to install . . . 13

4.2 Environment variables (.env) . . . 13

4.3 Ports reference . . . 14

4.4 Manual setup notes & gotchas . . . 14

5. End-to-End Testing — Scenario Playbook 16

5.1 Preflight — build your own world (shared by all scenarios) . . . 16

5.2 Scenario 1 — Greenfield, NO external integrations . . . 17

5.3 Scenario 2 — Greenfield, WITH external integrations . . . 17

5.4 Scenario 3 — Brownfield, uploaded resources only (no external tools) . . . 18

5.5 Scenario 4 — Brownfield, WITH external tools . . . 19

5.6 Scenario 5 — Interactive clarifications & offline takeover (cross-cutting) . . . 19

5.7 If something stalls . . . 20

1. What We Are Building — and What We Are Trying to Achieve

The product is not “an AI that writes code.” It is a platform that runs your entire software lifecycle the way a disciplined engineering organisation does — with specialists, evidence, review gates, and an audit trail — but with AI agents doing the heavy lifting and humans staying in control of every decision that matters.

1.1 The ultimate goal

We already had a powerful asset: a suite of **17 expert AI agents** (ideation, requirements, PRD, architecture, UX, task-breakdown, implementation, code review, testing, security, CI/CD, deployment, and more) plus the rules and gates that made them behave like senior engineers. The catch: that suite ran *locally*, glued to one developer's machine and Git checkout, with project context living in Markdown files.

The goal of this platform is to transform that suite into a **multi-tenant, web-delivered, engine-agnostic agentic SDLC platform** that:

- operates each client's **real tools of record** — Confluence, Jira, Figma, Xray, GitHub — bidirectionally, instead of inventing parallel copies;
- runs the **full lifecycle** (idea -> requirements -> PRD -> architecture -> UX -> build -> review -> test -> security -> CI/CD -> deploy) for both new (greenfield) and existing (brownfield) projects, single- or multi-repo;
- keeps a **human review gate** at every state change — nothing reaches a system of record without an approval;
- is **durable** (a run survives crashes, restarts, and an offline laptop) and **reproducible** (every run records the agent version, model, inputs, outputs, cost, and approver);
- treats the AI **engine as swappable** — Claude today, others later — without rewriting a single agent.

> **In one sentence**
>
> Take the intelligence of 17 senior-engineer AI agents, lift it off the laptop into a governed cloud-ready platform, point it at the client's real tools, and let teams ship software faster without giving up control, traceability, or quality.

1.2 The seven non-negotiables (the rules the whole system obeys)

Every design decision serves these locked principles. They are worth internalising because they explain *why* the platform is shaped the way it is.

#	Principle	What it means in plain terms
1	State vs. content	The platform owns **state** (pointers, provenance, events in Postgres). External tools own **content** (the actual Jira issue, Confluence page, Git PR). Local files are throwaway.
2	Orchestration vs. execution	The platform decides *what* happens and *when* (and runs the gates). The AI engine only *executes* a step. So engines stay interchangeable.
3	Memory = index, not copies	We store pointers + hashes + versions of artifacts, never copied bodies. This is what lets us detect drift and cross-check conflicts.

#	Principle	What it means in plain terms
4	Work is a graph	The unit of work is a **WorkItem**; work is a per-item DAG, not a rigid linear pipeline — features can progress independently.
5	Every artifact is gated	draft -> reviewed -> gated -> published. No auto-publish, ever. Nothing reaches a system of record before an approval.
6	Tenant isolation by RLS	Every database table carries a tenant_id and a Postgres Row-Level-Security policy from its first migration. One client can never see another's rows.
7	Capabilities, not vendors	Business logic calls tasktracker.* / docstore.* / codehost.* — never jira.* directly — so a connector can be swapped by editing one row.

Three more locked decisions round these out: the default engine is **Claude (Agent SDK)** behind an ExecutionDriver interface; durability is provided by **Temporal** from day one; and the agents are migrated by the **75 / 15 / 10** rule (reuse ~75% of an agent's reasoning verbatim, rewrite ~15% of its data references to the platform, replace ~10% of invocation glue).

1.3 The architecture at a glance — seven planes

The platform is organised into seven cooperating layers. You will meet each of these in the phase descriptions and again in the testing guide.

Plane	Responsibility	Where it lives
Experience	What people see and click: project workspace, phase console, gate approvals, traceability viewer.	React app (frontend/)
Control	The brain: Work-DAG state machine, Gate/Approval service, run scheduler, Manifest Composer, reconciliation, driver router.	Django services (backend/)
Execution	Runs the agents: the ExecutionDriver interface + ClaudeDriver; planning runs server-side, build runs on a developer-machine companion.	drivers/, runner/
Integration	Talks to the outside world through capability interfaces + connectors + the Platform-State MCP server + webhooks + a rate governor.	backend/integrations/, mcp_platform_state/
Memory / State	Postgres (with pgvector) + an S3-compatible object store: project profile, artifact registry, traceability graph, work DAG, event ledger, run history.	Postgres + MinIO
Security & Governance	Tenant isolation (RLS), prompt-injection defences, secret scanning, egress control, audit. (Deepens in Phase 5.)	across the stack
Observability & Quality	Durable workflow visibility, run cost metering, and the test/quality plane. (Deepens in Phase 5.)	Temporal + apps/quality/

1.4 What “done” looks like at each milestone

The build proceeds phase by phase, each ending in a working **thin vertical slice** demonstrated end-to-end. The headline milestone:

First usable product — after Phase 2

A user can take a **greenfield** project from idea -> PRD -> architecture -> backlog in Jira -> an implemented GitHub PR, and can **onboard an existing single-repo** project (from local documents or live Atlassian). Phases 3–4 add design/test/security depth and brownfield-full/multi-repo; Phases 5–6 add enterprise hardening, the cloud runner fleet, and engine choice.

2. The Build, Phase by Phase (Phases 0–4)

Each phase below lists the concrete components that were built and explains what each one does in everyday language, and closes with a **capstone slice** — the thin end-to-end demo that proves the phase works. (For a real, straight-through test pass — greenfield and brownfield — see the two journeys in **Section 5**.) Phases 0 through 4 are complete and tested.

2.1 Phase 0 — The Platform Spine

Goal: move the source of truth off the filesystem and run the existing agents through a real platform with state, gates, and provenance — before adding any external tools. This phase builds the skeleton everything else hangs on.

Component	What it is / does
State plane (Postgres)	The durable memory: Tenant, Project, **Project Profile** (the database record that replaces the old CLAUDE.md), Repository, **WorkItem** (the unit of work + its DAG), **Artifact Registry** (pointers + hashes + versions, never bodies), **Event Ledger** (append-only history), **Run History**, AgentSpec Registry.
Platform-State MCP server	A small tool server the agents call to read/write platform state: get_context, register_artifact, log_event, add_trace_edge, list/transition_work_item, etc. Every call is tenant-scoped.
Manifest Composer	Assembles a fresh, budget-bounded AGENTS.md per run from the Project Profile + relevant artifact pointers + recent events + rules. This is the context an agent reads — generated each run, never hand-maintained.
Temporal AgentRunWorkflow	The durable orchestration of one agent run: compose context -> execute agent -> verify it wrote what it should -> open the gate -> wait for a human -> publish + advance. Survives restarts; retries are idempotent.
Gate / Approval service	Gates are control-plane Approval objects, never engine hooks. A human with the right role decides; the decision signals the workflow to resume. (Phase 3 added WCAG checklist conditions to this.)
ExecutionDriver + ClaudeDriver	The seam that makes the engine swappable. Business logic calls the interface; the ClaudeDriver runs the Claude Agent SDK underneath.
Tenant isolation (RLS)	A non-superuser database role the app connects as, plus a Row-Level-Security policy on every table. Unset tenant context fails closed (sees no rows).
React Experience plane	The web UI: create a project, edit its profile, a Phase Console to trigger phases and watch the run timeline, a gate-approval screen, an artifact view, and a traceability view.
Migrated agents 01–04	Ideation, Requirements, PRD, Architecture — the first agents migrated by the 75/15/10 rule, plus the rules and per-agent triggers that replace the old slash commands.

> **Phase 0 — capstone slice (what this phase proves)**
> Run ideation -> requirements -> PRD -> architecture fully on platform state; artifacts registered as drafts in object storage; every step gated by a human; runs attributed; the manifest rendered per run. (This is exactly what you will click through in the testing guide, Section 3.)

2.2 Phase 1 — First System of Record (Atlassian)

Goal: make the platform operate a client's real Confluence and Jira, bidirectionally, with progress tracking and traceability — and let existing projects be onboarded.

Component	What it is / does
DocStore connector (Confluence)	Creates/updates Confluence pages (versioned, read-through). The platform publishes a PRD page only after its gate is approved.
TaskTracker connector (Jira)	Creates epics/stories/subtasks, transitions issues, comments, searches — headless via API token.
Capability abstraction	docstore.* and tasktracker.* interfaces + a per-tenant get_capability() selector. Callers never name Jira or Confluence directly (non-negotiable #7).
Artifact publish path	The gate -> publish -> pointer+version lifecycle to the live SoR; idempotent; falls back to “internal” when no connector is configured.
Traceability graph + viewer	requirement -> PRD -> epic -> story lineage, with deep-links into the live Confluence/Jira items.
Reconciliation service	Turns proposed work into accepted SoR issues without ever duplicating something that already exists (register-don't-recreate).
Brownfield-lite onboarding	Adopt an existing project: register what is already in Atlassian, and extract structured artifacts from static **Word/PDF/Excel** documents (with provenance + a review gate).
Rate governor	A per-(tenant, capability) token bucket + backoff + short-lived read cache, so live API calls stay within limits and don't flake.
Migrated agent 06	Task Breakdown — decomposes the PRD/architecture into a vertically-sliced backlog that reconciliation pushes into Jira on approval.

2.3 Phase 2 — Build Execution (real code, real PRs, durably)

Goal: produce real code — implementation, review, test — as GitHub pull requests, durably, *without changing how a developer works day to day*.

Component	What it is / does
CodeHost connector (GitHub)	Owns the PR / branch / status / comment / merge surface via the GitHub REST API. The engine never calls Git directly; the platform does, behind the codehost.* capability.
Local-companion runner	A small daemon a developer runs on **their own machine**. It enrolls with the platform, pulls a build task, and works in a clean, platform-managed **git worktree** (a feature/<key> branch) so the developer can run and test the change locally before any PR. Their main checkout is never touched.
Remote Platform-State serving	The companion uses the state tools over an authenticated remote transport — the dev machine needs no database credentials; tenant binding and RLS stay server-side.

Component	What it is / does
run_build_task	The build-mode driver call: launches the agent in the worktree with file + shell tools, under a per-phase tool allowlist (no CodeHost, no arbitrary network).
Temporal BuildWorkflow (saga)	Durable build orchestration: dispatch to the companion -> await its report (or park for human takeover if the laptop is offline) -> open a draft PR -> review -> remediation loop -> reviewer merge gate -> merge. Idempotent; compensating on failure.
07 <-> 08 remediation loop	Implementation (07) and Code Review (08) cooperate: each blocking finding becomes a child WorkItem; 07 re-runs in remediation mode until 08 is clean — then the merge gate can be granted.
Injection guardrails v1	Secret-scan before push (blocks credentials reaching GitHub), an egress allowlist (only approved hosts), and the per-phase tool allowlist.
Human takeover + webhook reconcile	A developer can take a WorkItem over and just work by hand; a signed GitHub webhook reconciles their commits/PR/CI status back into the registry (register-don't-recreate).
Light multi-repo	One project can target several repositories (one repo per WorkItem); each builds on its own branch/PR independently.
Migrated agents 07, 08, 11, 12	Implementation, Code Review, CI/CD, Deployment — with their reasoning (Clean Code/TDD, the eleven review dimensions, the Jenkins/Actions templates, blue-green/canary releases) preserved verbatim.

Phase 2 — capstone slice (the stop-and-dogfood MVP)

WorkItem -> companion worktree -> local build/test -> PR -> CI status -> review -> remediation -> reviewer merge gate -> merge, with a forced mid-run failure recovered idempotently, a human commit reconciled via webhook, and two repos in one project building independently.

2.4 Phase 3 — Design & Test Loops

Goal: close the UX and QA loops — read a client's live design, push tests and results to Xray, and turn a failing test into a tracked, linked bug.

Component	What it is / does
DesignSource connector (Figma)	**Read-as-SoR**: the platform reads the client's *current* Figma design at run time (frames, design tokens, components) rather than regenerating it. Canvas write-back is a deferred seam.
Design-System / tokens + Code Connect	Aggregates the current design into a design_system artifact (tokens) and a code_connect mapping that links design components to code components.
a11y (WCAG) gate	The accessibility review is enforced as a **checklist of WCAG 2.1 AA conditions on the design Approval** — an approve is blocked until every required criterion is satisfied. It is a control-plane gate, not a prompt rule.
Per-tenant theming	Design tokens + responsive breakpoints stored on the Project Profile flow verbatim into the manifest — one theme, no per-tenant code fork.
TestMgmt connector (Xray)	Authors test cases as Xray tests and imports execution results (idempotently). Headless, per-tenant, rate-governed.
Result import + defect loopback	A test run records a TestExecution + a test_report artifact; a **failing test opens a linked, traced defect** (a child WorkItem tied to the test and traced to the code); a later pass closes it.

Component	What it is / does
Test-impact selection	Given a changed piece of code, walks the traceability graph to select just the tests that cover it.
Flaky quarantine	A test that flips pass/fail within one run is quarantined and excluded from the blocking gate (never silently blocks a merge, never silently passes).
Ephemeral preview env per PR	A companion-local seam: the app is brought up from the worktree, the URL is posted on the PR, and torn down on close. (Cloud preview is a Phase-5 drop-in behind the same seam.)
Migrated agents 05, 09, 09b, 10	UX Design (read-as-SoR + HTML wireframes), Testing, Test Automation (Playwright-MCP locator discovery + Page Object Model preserved), and Security (the nine review tracks + SBOM, with criticals opening defects).

Phase 3 — capstone slice (what this phase proves)

Design produced and gated through the WCAG checklist -> implementation reads the current design -> tests author to Xray and results import -> a failing test opens a traced defect that closes on a clean re-run -> a standalone security review -> a companion-local preview env. Verified end-to-end, with the full platform suite green across all phases.

2.5 Phase 4 — Brownfield-Full & Multi-Repo

Goal: the client front door. Onboard an *existing* project — any number of repositories — understand its code, **resume the lifecycle from the middle** instead of starting over, and keep cross-repo API contracts honest with **live drift detection**. This is the entry point most real clients will use.

Component	What it is / does
CodeIntel capability + built-in analyzer	A fixed 8-tool codeintel.* surface with a **built-in SOTA analyzer** (no license needed): git/Tornhill **hotspots** (churn × complexity), a dependency graph with **PageRank** + communities, dead-code and complexity, and decision archaeology. It runs in the **companion worktree** on a real clone; the platform stores only the index + artifacts, **never the source**. Repowise (or another engine) plugs in behind the same seam.
Knowledge Index	An RLS-aware chunk store with an always-on, **zero-cost keyword (BM25) baseline** plus an optional embedding seam (Voyage / Gemini). It feeds the manifest's retrieved-knowledge section and is **project-wide shared memory** across all of a project's repos.
Agent 00c — Intelligent Context Extractor	Migrated: turns a raw codebase into five intelligence layers (dependency graph, git analytics, dead code, decisions, health) registered as platform artifacts + indexed into the Knowledge Index. tech_lead gate.
Agent 00 — Onboarding (full)	The brownfield front door: **discover** across the connected systems of record + code-intel, **register** what already exists (register-don't-recreate), **reconcile**, detect the **gaps**, and emit a **gap report** that assesses every lifecycle phase Complete/Partial/Missing and names the correct **resume phase**. product_owner gate.
Agent 00b -> API-Contract artifact	Cross-repo contracts as a **versioned api_contract registry artifact** (read-through, never a copied file): the provider repo exports it (semver-bumped); consumer repos read the current version at run time and record an implements edge. architect gate. (Agent 00d Snapshot is retired — continuity is now the database + durable runs.)

Component	What it is / does
Multi-repo project model + per-repo manifest	One project owns **N repositories** (frontend/backend/service); topology reconciles to single/split/multi. The composed manifest is **scoped per repo** (that repo's artifacts + project-shared ones + the shared contract), while the registry, traceability, and Knowledge Index stay **project-wide shared memory**.
Cross-repo DAG edges	Dependencies that span repositories: implements traceability edges across repos + WorkItem.depends_on — the basis for cross-repo change-impact.
Live drift detection + drift gate	Two detectors: **contract drift** (an api_contract diverges from its consumers) and **pointer/hash drift** (a SoR artifact's external content changed since we indexed it). On detection the platform writes a **DriftRecord**, computes **change-impact via the traceability graph** (every downstream consumer/repo), and opens a decide_drift gate — architect for contract, tech_lead for pointer/hash. **Accept** re-indexes; **reject** flags for revert; nothing is silently written to a system of record. The signed Git push webhook is the live trigger (built on the Phase-2 reconcile).

Phase 4 — capstone slice (what this phase proves)

Onboard a **split FE/BE** project -> register-don't-recreate + the built-in code-intel running for real in a companion worktree -> a **gap report** with a resume phase -> the backend exports an api_contract, the frontend reads it read-through and records a cross-repo implements edge -> **resume mid-pipeline** triggers the next phase to its gate -> a **contract change opens a drift gate** whose change-impact **spans both repos**, decided by the architect. Verified end-to-end; the full platform suite is green across all phases.

3. The Local Execution & Testing Guide

This is a hands-on guide for a developer to run the platform locally and *see* every workflow work — in the browser, the API, the Temporal dashboard, and the object store. It is not about unit tests. Follow Part A to get the core running with zero external accounts; Part C adds the optional live connectors.

Before you start

Complete the prerequisites in **Section 4** first (Docker Desktop running, .env created). **Agents run on your own Claude Code** (req #1): every phase — planning, build, and review — executes on the machine running the **companion** daemon, using your **logged-in local claude** (your subscription), **not** an API key. So instead of setting ANTHROPIC_API_KEY, you log in once (claude / claude setup-token) and run the companion (Step 4). A server-side ANTHROPIC_API_KEY path still exists as a headless/CI fallback only. On Windows without make, use ./make.ps1 <target>.

3.1 Part A — Bring up the platform (no external accounts needed)

Step 1 — Start the infrastructure

```
# from the repo root
docker compose up -d        # postgres (:5433), temporal (:7233 + UI :8080), minio (:9000 + console :900
docker compose ps              # all services should be 'running' / healthy
```

What you should see: open the **Temporal UI** at http://localhost:8080 (empty workflow list for now) and the **MinIO console** at http://localhost:9001 (login minioadmin / minioadmin; the sdlc-artifacts bucket already exists).

Step 2 — Migrate the database and seed the demo tenant

```
make migrate     # creates every table + its RLS policy (runs as the DB owner)
make seed        # seeds a demo tenant, project, profile, agent specs, connectors
```

What you should see: seed prints the created tenant/project ids and a connector count. (If make migrate reports an ownership error on an ALTER, run it as the postgres superuser — see Section 4.4.)

Step 3 — Run the application (API + worker + frontend)

```
make dev         # = docker compose --profile app up
#   backend API  -> http://localhost:8000
#   React UI     -> http://localhost:5173
#   worker       -> connects to Temporal (durable runs execute here)
```

What you should see: the React app loads at http://localhost:5173; the worker logs “started” and registers with Temporal. A quick API smoke check:

```
curl http://localhost:8000/api/projects/ -H "X-Tenant-ID: <demo-tenant-id>"
# -> JSON list including the seeded demo project
```

Step 4 — Run your local companion (so phases execute on YOUR Claude Code)

Every phase the platform dispatches runs on the machine running the companion, against your logged-in claude — no API key. Log in once, enroll this machine for your user, then leave the daemon running:

```
# 1) log Claude Code in once (interactive, or a long-lived token for headless):
claude            # sign in to your subscription   (or: claude setup-token)

# 2) enroll this machine for YOUR user (prints a one-time token):
python -m runner.companion enroll --api-url http://localhost:8000 \
    --tenant $TENANT --project $PROJECT --label my-laptop --developer-id $USER

# 3) run the headless daemon — it claims your dispatched phases and runs them locally:
python -m runner.companion serve --api-url http://localhost:8000 \
    --tenant $TENANT --token <enrollment-token>   # add --repo-path/--worktrees for builds
```

What you should see: the daemon logs that it preflighted claude on PATH (and dropped any ANTHROPIC_API_KEY so your subscription is used), then idles polling for work. When you trigger a phase (next), it claims and runs it; an agent that needs input surfaces a question on the **Clarifications** tab and waits at no cost.

3.2 Part B — Drive the planning slice in the browser (Phase 0 end-to-end)

This is the core, no-external-accounts workflow. Open http://localhost:5173 and work left to right through the tabs. With your companion from Step 4 running, each triggered phase executes on your local Claude Code.

Cost & interaction note

Phases run on your **local Claude Code subscription** via the companion — there is no per-call API charge and no ANTHROPIC_API_KEY needed. If an agent needs information it cannot find, it asks on the **Clarifications** tab and **pauses at zero cost** until you answer (the model is idle while it waits) — then continues in the same session. To see the whole loop quickly, trigger a single phase (Ideation).

Tab / action	What to do	What you should see (the proof)
Tenant + Project bar	Create or select a tenant, then a project (the seeded demo is fine).	The tabs unlock.
1. Project Profile	Fill description, tech stack, important context; Save.	A new profile version is saved; it becomes the agent's context next run.
2. Phase Console	Add a WorkItem (a feature stream), then click **Ideation**.	A Run appears in the Run Timeline and moves running -> awaiting_gate as your companion executes it. In the Temporal UI (:8080) a new AgentRunWorkflow is visible. A **"needs answer"** badge appears if the agent asks a question.
3. Clarifications	If a run shows "needs answer", open this tab, answer the question(s), and Send.	The agent resumes in the same session (it was paused at zero cost); the badge clears.
4. Gate Approvals	Find the pending approval; approve it as a user holding the required role.	The run resumes to completed; the workflow finishes in the Temporal UI.
5. Artifacts	Look at the ideation artifact.	Its status flips draft -> published; its body is stored in MinIO (visible in the :9001 console under drafts/ and snapshots/).
6. Traceability	Open the lineage view.	A graph of artifacts and their derives/tests/... edges; once external SoRs are connected, nodes deep-link to the live Confluence/Jira items.

Repeat the trigger for **Requirements -> PRD -> Architecture** to walk the full planning chain. Each run reads the artifacts the previous phase published (proving the platform — not a local file — is the source of truth), and each is independently gated.

3.3 Part C — Optional: exercise the live connectors

These steps need real (test) cloud accounts. Each is independent — do only the ones you have credentials for. Set the relevant variables in .env (Section 4) and make seed again so the demo tenant gets the connector, then restart make dev.

Atlassian — Confluence + Jira (Phase 1)

- Set ATLASSIAN_*, CONFLUENCE_SPACE_KEY, JIRA_PROJECT_KEY.
- **Greenfield:** approve the PRD gate -> the PRD **publishes as a Confluence page**; approve the backlog -> epics/stories **appear in Jira**. The Traceability tab now deep-links to those live items.
- **Brownfield:** POST a Word/Excel document to /api/source-documents/ -> it is extracted into reviewable internal artifacts (gated before adoption); existing Jira issues are registered without duplicates.

GitHub — the build path (Phase 2)

Set GITHUB_API_URL, GITHUB_TOKEN (a PAT with repo scope), GITHUB_REPO ("owner/name"), GITHUB_DEFAULT_BASE, GITHUB_WEBHOOK_SECRET. Then enroll a companion on your machine and run a build:

```
# 1) enroll this machine as a runner (prints a one-time token)
python -m runner.companion enroll --api-url http://localhost:8000 \
    --tenant <tenant-id> --project <project-id> --label my-laptop --repos owner/name

# 2) run an assigned build task in a managed worktree
python -m runner.companion run --task task.json \
    --repo-path ~/repos/your-clone --worktrees ~/sdlc
```

What you should see: a feature/<key> branch is created in a worktree, tests run locally, a **draft PR opens on GitHub**, code review runs, and the Temporal UI shows the BuildWorkflow parked at the **reviewer merge gate**. Approve it (Gate Approvals) and the PR **merges**. Push a manual commit to the branch and the signed webhook reconciles it (the artifact's sha pointer moves; no duplicate).

Figma + Xray (Phase 3)

- **Figma:** set FIGMA_TOKEN + FIGMA_FILE_KEY (a personal-access token + a test file). The design phase reads the *current* design tokens/components from that file (read-as-SoR) instead of inventing them.
- **Xray:** set XRAY_CLIENT_ID + XRAY_CLIENT_SECRET + XRAY_PROJECT_KEY. The test phase authors tests into Xray and imports results; a failing import opens a defect WorkItem linked to the test and traced to the code, which closes automatically on a clean re-run.

3.4 Part D — See the cross-cutting guarantees with your own eyes

Guarantee	How to observe it
Durability	In the Temporal UI (:8080), open a running workflow — it shows each activity, retries, and the gate wait. Restart the worker container mid-run; the workflow resumes where it left off.
Gate-then-publish	On the Artifacts tab, an artifact stays draft until you approve its gate — then it becomes published. There is no way to publish without an approval.
Tenant isolation (RLS)	Create a second tenant and call the API with its X-Tenant-ID: it sees none of the first tenant's projects, runs, artifacts, or defects.
Provenance / cost	The Run Timeline shows each run's engine, model, cost (USD), and Temporal workflow id; the Event ledger records who approved what and when.
Artifact bodies in S3	The MinIO console (:9001) shows artifact draft bodies and immutable post-approval snapshots — the registry holds only pointers + hashes.

"Everything works" checklist

(1) infra healthy; (2) migrate + seed succeed; (3) UI loads and the tabs work; (4) a planning phase runs to completed through its gate and publishes an artifact; (5) traceability shows the lineage; (6) the Temporal UI shows the durable workflow; (7) MinIO shows the artifact body; (8) optional: a live PRD reaches Confluence, a backlog reaches Jira, a build reaches a GitHub PR, and a failing test opens a linked defect.

4. Setup, Credentials & Prerequisites

Everything a developer must install, configure, or obtain before running Section 3. Copy .env.example to .env and fill the values below.

4.1 Prerequisites to install

Tool	Why	Notes
Docker Desktop	Runs the whole stack (Postgres, Temporal, MinIO, API, worker, frontend).	Must be **running** before docker compose up. Windows: WSL2 backend recommended.
Python 3.12 + a repo-root .venv	Local tooling: fast pytest/ruff, the companion runner CLI.	python -m venv .venv then install backend/requirements-dev.txt. Tooling only — the app runs in Docker.
Git	Version control + the companion's managed worktrees.	Any recent version.
make (optional)	Convenience targets.	Windows without make: use ./make.ps1 <target>.

4.2 Environment variables (.env)

Required for the core local run

Variable	Purpose	Default / value
DJANGO_SECRET_KEY	Django signing key (dev).	any dev string
POSTGRES_* / DATABASE_URL	App database + the non-superuser RLS role.	preset in .env.example (app uses sdlc_app; host port 5433)
TEMPORAL_HOST / NAMESPACE / TASK_QUEUE	Durable workflow engine.	temporal:7233 / default / sdlc-planning
CORS_ALLOWED_ORIGINS	Lets the React app call the API.	http://localhost:5173
S3_* (MinIO)	Object store for artifact bodies + snapshots.	minioadmin / minioadmin, bucket sdlc-artifacts
ANTHROPIC_API_KEY	**Required to actually run agents** (the ClaudeDriver). Each run costs a little.	sk-ant-... from console.anthropic.com
DEFAULT_PLANNING_MODEL	Which Claude model planning runs use.	claude-opus-4-8 (use a cheaper model to reduce cost)

Optional — only for the live connector walkthroughs (Part C)

Variable group	Enables	Where to get it
ATLASSIAN_SITE_URL, ATLASSIAN_EMAIL, ATLASSIAN_API_TOKEN, ATLASSIAN_CLOUD_ID, CONFLUENCE_SPACE_KEY, JIRA_PROJECT_KEY	Phase 1: PRD -> Confluence, backlog -> Jira, traceability deep-links, brownfield onboarding.	API token: id.atlassian.com -> Security -> API tokens. Use a TEST space/project.

Variable group	Enables	Where to get it
GITHUB_API_URL, GITHUB_TOKEN, GITHUB_REPO, GITHUB_DEFAULT_BASE, GITHUB_WEBHOOK_SECRET	Phase 2: companion build -> draft PR -> review -> merge gate -> merge; webhook reconcile.	A PAT with repo scope (github.com -> Settings -> Developer settings). Add these keys to .env (not in the template by default).
FIGMA_TOKEN, FIGMA_FILE_KEY	Phase 3: read the current design as the source of truth.	Figma -> Settings -> Personal access tokens; a test file's key from its URL.
XRAY_CLIENT_ID, XRAY_CLIENT_SECRET, XRAY_PROJECT_KEY	Phase 3: author tests + import results; failing test -> linked defect.	Xray Cloud -> Global Settings -> API Keys. Use a TEST project.

Security

Real credentials live only in .env, which is git-ignored — never commit them. Always use **test** spaces/projects/repos, not production ones. Without any connector creds the core local run (Parts A & B) still works fully on the internal source of truth.

4.3 Ports reference

Service	URL	Login
React UI (Experience plane)	http://localhost:5173	—
Backend API	http://localhost:8000/api	X-Tenant-ID header
Temporal Web UI	http://localhost:8080	—
MinIO console (object store)	http://localhost:9001	minioadmin / minioadmin
Postgres (host-mapped)	localhost:5433	postgres / postgres (admin)

4.4 Manual setup notes & gotchas

- **Migrate as the DB owner.** The app connects as the non-superuser sdlc_app role (so RLS applies). Schema changes (ALTER) must run as the postgres superuser. If make migrate fails on an ALTER, run migrations with the superuser DSN: DATABASE_URL=postgres://postgres:postgres@localhost:5433/sdlc_platform.
- **Postgres is on host port 5433**, not 5432, to avoid clashing with a native Postgres. In-container services still use postgres:5432.
- **If the stack drops** (a “Socket is not connected” error on :5433 usually means Docker Desktop stopped), start Docker Desktop and run docker compose up -d again.
- **Windows without make:** use ./make.ps1 <target> (e.g. ./make.ps1 dev).
- **First frontend start** runs npm install inside the container — the very first make dev takes a few minutes while node modules download.
- **The companion runner** runs on the developer's own machine (not in Docker) via python -m runner.companion ...; it connects outbound to the platform API.

Quick start, condensed

cp .env.example .env (set ANTHROPIC_API_KEY) -> docker compose up -d -> make migrate -> make seed -> make dev -> open http://localhost:5173 and run a phase through its gate.

This document describes Phases 0–4 (complete and tested). Section 5 is an explicit test-through inventory of what works today. The next milestone is Phase 5 — Enterprise Hardening & Cloud Migration (RBAC + segregation of duties, secrets vault, prompt-injection hardening, immutable audit/compliance, the eval harness, and the cloud runner fleet).

5. End-to-End Testing — Scenario Playbook

This section is a hands-on playbook for an engineer to test the platform across the scenarios that matter, **each built from scratch** — you create your own tenant, user, project and companion; nothing here uses the seeded demo tenant. Do **Section 3** bring-up once, then the shared **5.1 preflight**, then run whichever scenarios you need. Each scenario lists every step and whether it is done in the **UI** or via the **API** (used only where no UI exists yet).

> **Two ground rules for every scenario**
>
> (1) **From scratch:** you create the tenant/user/project yourself — do not use the seed demo tenant.
> (2) **Agents run on your Claude Code:** every phase executes on your machine via the **companion** daemon using your logged-in claude (no API key); you only ever click and type in the web app. API calls carry -H "X-Tenant-ID: $TENANT" (tenant-create is the one exception).

5.1 Preflight — build your own world (shared by all scenarios)

Step 1 (UI/infra). Bring the stack up once (Section 3.1): docker compose up -d -> make migrate -> make dev. You do **not** run make seed — you are building everything yourself.

Step 2 (API) — create your tenant. Creating a tenant also seeds that tenant's agent specs (all phases), so it is immediately usable:

```
export API=http://localhost:8000/api
curl -s -X POST $API/tenants/ -H Content-Type:application/json -d '{"name":"My Co"}'
export TENANT=<id-from-response>
export H="-H X-Tenant-ID:$TENANT -H Content-Type:application/json"
```

Step 3 (API/shell) — create your user and grant the gate roles. There is no user-admin UI yet, so create the user + memberships once via a marked shell call, and capture the user id (used to answer clarifications and decide gates):

```
docker compose exec backend python manage.py shell -c "\
from apps.tenancy.models import User, Membership; from apps.projects.models import Project;\
u,_=User.objects.get_or_create(email='me@myco.test', tenant_id='$TENANT', defaults={'name':'Me'});\
u.set_password('pw'); u.save(); print('USER', u.id)"
export USER=<id-printed-above>
# (after you create a project in each scenario, grant every gate role on it:)
# docker compose exec backend python manage.py shell -c "...Membership.get_or_create(
#   tenant_id='$TENANT', user_id='$USER', project_id='$PROJECT', role=r) for r in [...]"
```

Step 4 (CLI) — log in Claude Code and run your companion. This is what makes phases run on your machine. Do it once per scenario's project (or pass several repos):

```
claude            # sign in to your subscription   (or: claude setup-token for headless)
python -m runner.companion enroll --api-url http://localhost:8000 \
    --tenant $TENANT --project $PROJECT --label my-laptop --developer-id $USER
#   -> prints a one-time enrollment token; then leave the daemon running:
python -m runner.companion serve --api-url http://localhost:8000 \
    --tenant $TENANT --token <enrollment-token>      # add --repo-path/--worktrees for builds
```

How a gate or a question is handled (the same in every scenario)

When a run reaches a human gate it appears on the **Gate Approvals** tab — approve as $USER (who must hold that gate's role). If an agent needs information it posts a question on the **Clarifications** tab and the run shows **“needs answer”** in the Phase Console; answer it and the run continues at no extra cost. API equivalents: POST /approvals/{id}/decide/ and POST /clarifications/{id}/answer/.

5.2 Scenario 1 — Greenfield, NO external integrations

Prove the full **planning/doc lifecycle** with everything kept internal — no Confluence/Jira/Figma/Xray. Artifacts publish to the platform's internal store (bodies in MinIO).

- **(API) Create the project:** curl -s $H -X POST $API/projects/ -d '{"name":"Acme Shop","track":"greenfield"}' -> export PROJECT=<id>. Then grant roles (5.1 Step 3) and enroll+serve the companion (5.1 Step 4).
- **(UI) Project Profile tab:** fill description, tech stack, important context; Save.
- **(UI) Phase Console tab:** add a WorkItem (feature stream), then trigger phases in order: **Ideation -> Requirements -> PRD -> Architecture** (and 05/06 if you wish). Each runs on your companion.
- **(UI) Clarifications tab:** if a phase shows “needs answer”, answer it -> the run resumes in the same session.
- **(UI) Gate Approvals tab:** approve each phase's gate as $USER -> the run completes and its artifact flips draft -> published.
- **(UI) Artifacts + Traceability tabs:** see each artifact (body in the MinIO console :9001) and the lineage graph (derives edges) — all sor=internal.

Scenario 1 proof

A greenfield project went idea -> requirements -> PRD -> architecture entirely on your local Claude Code, each phase human-gated, every artifact published internally with full lineage — and **zero external accounts**. (The build/merge + live-SoR phases are Scenario 2.)

5.3 Scenario 2 — Greenfield, WITH external integrations

Same lifecycle as Scenario 1, but real systems of record: PRD -> Confluence, backlog -> Jira, UX from live Figma, build -> a GitHub PR, tests -> Xray. Set the credentials in .env (Section 4.2) and **attach the connectors to YOUR tenant** (the seed does this only for the demo tenant), then restart make dev:

```
# attach connectors to your from-scratch tenant (creds resolve from .env):
docker compose exec backend python manage.py shell -c "\
from django.conf import settings as s; from apps.integrations.models import IntegrationConnection as C;\
rows=[('docstore','confluence',{'space_key':s.CONFLUENCE_SPACE_KEY}),\
      ('tasktracker','jira',{'project_key':s.JIRA_PROJECT_KEY}),\
      ('codehost','github',{'repo':s.GITHUB_REPO,'default_base':s.GITHUB_DEFAULT_BASE}),\
      ('designsource','figma',{'file_key':s.FIGMA_FILE_KEY}),\
      ('testmgmt','xray',{'project_key':s.XRAY_PROJECT_KEY}),('codeintel','builtin',{})];\
[C.objects.update_or_create(tenant_id='$TENANT',capability=c,connector=n,defaults={'scopes':sc}) for c,n
```

Now run the lifecycle exactly as Scenario 1 (UI Phase Console + Clarifications + Gate Approvals). The same two moves per phase — **trigger**, then **approve** — now also write to the live tools. Per-phase agent

keys, gate roles, and the live proof:

Phase — agent_key	Gate role	Live proof on approval
PRD — 03_prd	product_owner	A **Confluence page** is created; the artifact's external_ref.url + the Traceability node deep-link to it.
UX Design — 05_ux_design	designer	Reads your **live Figma** tokens/components; the **WCAG gate** blocks approval until every required criterion is ticked.
Task Breakdown — 06_task_breakdown	tech_lead	**Epics/stories appear in Jira**; child WorkItems + story->requirement trace links are created.
Implementation — 07_implementation	tech_lead	Runs on your companion worktree -> a **draft PR opens on GitHub** (start the companion with --repo-path/--worktrees).
Code Review — 08_code_review	reviewer	Runs on your companion; findings become child WorkItems; the 07<->08 loop re-runs until clean.
Testing — 09_testing	qa_lead	**Tests author to Xray** + results import; a failing test opens a linked, traced defect that closes on a clean re-run.
Merge	reviewer	Approve the merge gate -> the **GitHub PR merges**; the WorkItem moves to done.

Scenario 2 proof

The same greenfield run now produced a **Confluence PRD**, a **Jira backlog**, **UX from live Figma** behind a WCAG gate, a **merged GitHub PR**, and **Xray tests with a linked defect** — each gated, each on your local Claude Code.

5.4 Scenario 3 — Brownfield, uploaded resources only (no external tools)

Onboard an existing project using only what you **upload** — no Confluence/Jira. The platform assesses what exists and tells you which phase to resume. Brownfield onboarding services have no dedicated UI yet, so the onboarding steps are API/shell; gates are approved in the UI. **Test onboarding at different stages** by varying what you provide:

- **(API) Create the brownfield project** + grant roles + enroll/serve the companion (5.1): curl -s $H -X POST $API/projects/ -d '{"name":"Legacy Shop","track":"brownfield"}'.
- **(API) Upload static resources:** curl -s -H "X-Tenant-ID:$TENANT" -F file=@./old-spec.docx $API/projects/$PROJECT/source-documents/ (repeat for .xlsx/.pdf). Each is extracted into reviewable draft artifacts.
- **(UI) Adopt them behind a gate:** approve adoption — or POST /source-documents/{id}/adopt/ {"user_id":"$USER"}.
- **(shell) Optional code intelligence** on a local clone (built-in analyzer; the platform stores the analysis + index, never your source): run BuiltinCodeIntel().index(workspace=...) + tools.register_artifact(...) (see the snippet in 5.5).
- **(shell) Gap report -> resume phase:** build_gap_report(tenant_id=$TENANT,project_id=$PROJECT) returns resume_phase + the blocking gaps.

- **(UI) Resume mid-pipeline:** trigger the phase it named (e.g. 04_architecture) on a WorkItem and approve its gate — onboarding hands straight into the normal pipeline.

> **Onboarding at different stages — vary the inputs**
>
> **Empty** (upload nothing) -> the gap report resumes at ideation. **Partial** (upload a requirements doc only) -> it resumes at prd/architecture. **Mature** (docs + code intelligence) -> it assesses implementation/testing as present and resumes later. Run the gap report after each upload to watch the resume point move.

5.5 Scenario 4 — Brownfield, WITH external tools

As Scenario 3, plus live tools: register the project's **existing** Confluence/Jira (register-don't-recreate — no duplicates), and exercise the multi-repo API-contract + live drift gate. Attach connectors to your tenant (5.3 snippet) first.

```
# 1) (shell) register repos (multi-repo) + (API) register existing Confluence/Jira
docker compose exec backend python manage.py shell -c "\
from apps.projects.repos import add_repository as A;\
A(tenant_id='$TENANT',project_id='$PROJECT',name='owner/web',role='frontend');\
A(tenant_id='$TENANT',project_id='$PROJECT',name='owner/api',role='backend')"
curl -s $H -X POST $API/projects/$PROJECT/onboard-atlassian/   # existing items, deduped

# 2) (shell) code intel on a clone -> gap report -> versioned API contract (read-through)
docker compose exec backend python manage.py shell -c "\
from integrations.onboarding import build_gap_report;\
from integrations.api_contract import export_api_contract, read_api_contract;\
print('RESUME ->', build_gap_report(tenant_id='$TENANT',project_id='$PROJECT')['resume_phase']);\
c=export_api_contract(tenant_id='$TENANT',project_id='$PROJECT',body='GET /users',source_commit='abc123'
```

```
# 3) (shell) breaking contract change -> drift gate with cross-repo impact -> architect decides
docker compose exec backend python manage.py shell -c "\
from integrations.api_contract import export_api_contract;\
from integrations.drift import detect_contract_drift, decide_drift;\
b=export_api_contract(tenant_id='$TENANT',project_id='$PROJECT',body='GET /users (v2)',change='major');\
d=detect_contract_drift(tenant_id='$TENANT',contract_artifact_id=b['artifact_id'],new_semver='2.0.0');\
print('impacted ->', d['impacted']);\
print(decide_drift(tenant_id='$TENANT',drift_id=d['drift_id'],user_id='$USER',decision='accept'))"
#   live alternative: push to the provider repo so the signed GitHub webhook opens the drift gate
```

(UI) Resume mid-pipeline at the gap-report phase and continue exactly as Scenario 2 (phases run on your companion; PRD/backlog/PR/tests hit the live tools).

> **Scenario 4 proof**
>
> An existing multi-repo project was onboarded with its real Confluence/Jira registered (no duplicates) and its code indexed, produced a **gap report + resume phase**, exported a **versioned API contract read across repos**, and a **breaking change opened a drift gate** whose impact spanned both repos.

5.6 Scenario 5 — Interactive clarifications & offline takeover (cross-cutting)

These prove the two Phase-4.5 guarantees directly and apply on top of any scenario above.

- **Zero-cost clarification (UI):** trigger any phase; if the agent asks, the Phase Console shows **“needs answer”** and the run sits idle (your claude is not generating, so nothing is billed). Answer on the **Clarifications** tab -> the run resumes in the same session.
- **Durability (no re-ask):** stop the companion (Ctrl-C) while a clarification is answered but the phase hasn't reported, then re-trigger the phase — the agent reuses your earlier answer (folded into its manifest) instead of asking again.
- **Offline takeover:** stop the companion, trigger a phase, and leave it; after the report timeout the run parks the WorkItem for **human takeover** (Run status failed + a takeover flag) rather than hanging — restart the companion to resume normal operation.
- **Local engine, not the API (verify):** the companion log shows it dropped ANTHROPIC_API_KEY and used claude on PATH; usage shows on your Claude subscription, not an API bill.

5.7 If something stalls

Almost always it is the environment, not the platform. Quick checks:

- **Infra down:** a “Socket is not connected” on :5433 means Docker Desktop stopped — start it and docker compose up -d again.
- **Migrations:** if make migrate errors on an ALTER, run it with the postgres superuser DSN (Section 4.4).
- **Phase never runs / stays pending:** your **companion isn't running** or isn't enrolled for $USER — check serve is up and you triggered with your user_id; confirm claude is logged in on that machine.
- **Live SoR step does nothing:** the connector row isn't attached to *your* tenant (run the 5.3 snippet) or the creds aren't in .env; restart make dev.
- **Gate won't approve (403):** $USER doesn't hold that gate's role on the project — re-run the role-grant in 5.1 Step 3.
- **Full automated proof anytime:** cd backend && pytest -q — including tests/test_acceptance_phase45.py (the local-execution + clarification loop) and tests/test_acceptance_phase4.py (brownfield/multi-repo).